# Hot Dog, the Autonomous AI-Powered Robot - EGN4060C Final Project

Anthony Terry, Diego Pedroza, Dylan Vidal, Richard Phillips

*University of Central Florida*
*4000 Central Florida Blvd., Orlando, FL 32816*

an994022@ucf.edu

di246213@ucf.edu

dylan.vidal@ucf.edu

richard.phillips2@ucf.edu

***Abstract*— This document gives a high level overview of Hot Dog, a differential drive robot based on the open-source Jetbot framework published by NVIDIA that uses an agentic system of LLMs to reason about and perform a variety of generalized tasks. This system was created by Anthony Terry, Diego Pedroza, Dylan Vidal, and Richard Phillips for the EGN4060C Intro to Robotics Course offered at the University of Central Florida.**

**This system was inspired by the rise in physical agents - robotic systems that leverage AI to perform generalized tasks that were not explicitly preprogrammed. While Hot Dog is a bit high latency, this system effectively accomplishes the task of truly generalized decision making and is only limited by the form factor of the robot host and the latency of LLM operations.**

***Keywords*— Robotics, LLMs, Agents, NVIDIA, Computer Vision**

## I. Introduction

In the last 5 years society has witnessed an explosion in the popularity, performance, and scalability of LLMs, and more formally, deep learning and other advanced methods. Following this rise came the exploration of generalized multi-model models in the domain of robotics, known as Vision Language Action models or VLAs, which train on multi-modal video/image data and output motor signals as tokens.

While this paradigm is limited by the availability of clean data, LLMs that create text as its output tokens are still useful for reasoning about various general concepts and making decisions. Thus, by allowing these text-based systems contextual understanding of the world around it via computer vision generated text labels, descriptive telemetry data, and an image feed, they can still prove useful for generalized robotic decision making - despite not being explicitly trained to do so.

## II. Method

### A. Pivot

As a disclaimer, our initial project proposal was an exploration incentivized Reinforcement Learning system that found and narrated novel view points. While this idea was exciting in the frontier of physical agents that are capable and efficient, it was outside the scope of our team's compute power. Instead, the introduction of LLMs pivoted the project idea from extreme precision at a specific task, to ultimate generalization across many non-deterministic and unseen tasks instead.

### B. Hardware / Jetbot Assembly

Our approach to implementing our generalized physical AI agent started at the hardware level. We began by researching mainstream open-source robotics projects as we are a team of Computer Science students and wanted an open platform with hardware documentation. Thus, we arrived at the Open Source Jetbot platform [1] built by NVIDIA.

At its core, the Jetbot is a differential drive robot assembled with accessible parts, 3D-printed components, and an NVIDIA Jetson Orin Nano single board computer. During assembly, we ran into issues with portable batteries that could support the 9V power delivery needed for the SPC and with voltage management for peripherals. Along with the power delivery issue we decided to include an ultrasonic sensor so that we could know the distance from an object. The problem was the ECHO (output pin) sent 5.5 V to the Jetson however, the pins can only take 3.3 V. To fix that issue we included a Hitletgo Bidirectional 5.5 - 3.3 converter to step down the voltage and used jumper wires to connect everything.

Beyond the differential drive, we also integrated camera data with an Arducam and proximity data via an ultrasonic sensor.

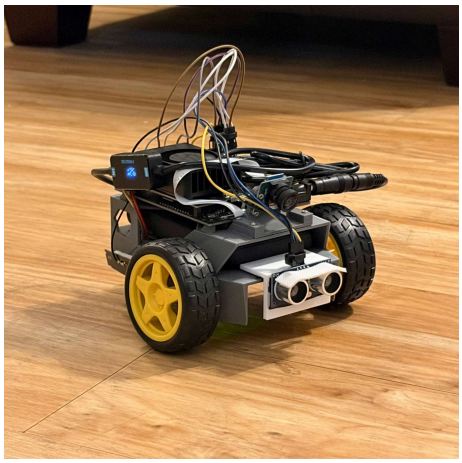

Fig. 1 Fully assembled final iteration of the Jetbot

### C. Edge / Jetbot Programming Tasks

After assembling the Jetbot, the project split into two primary subdomains - edge and remote compute. At the edge, we programmed the various control operations for the robot itself, containing the following primitive behaviors: `move_distance`, `rotate`, `move_arc`, `scan`, `center`, `queue_movement`. Each of these behaviors were manually calibrated with extensive real-world testing for the rotational speed, static friction coefficients. After implementing the abstract movement patterns, we exposed them to be teleoperated via an HTTP API (easily used by AI/Web interfaces) and tunneled the connection over SSH.

| **Programmed Edge Behaviors** | | |
|---|---|---|
| **Function** | **Parameters** | **Use** |
| move_distance | distance_m, speed | Drive straight. |
| rotate | angle_degrees, speed | Rotate direction. |
| move_arc | radius_m, angle_degrees, speed | Drive along a curve or turn pattern. |
| scan | labels | Uses CV to search for items and note found sectors. |
| center | label, center_threshhold, speed | Rotates until the target item is within the center threshold in view. |
| queue_movement | list[function_calls] | Executes a sequence of any other commands |
| start_movement | speed | Unbounded forward movement |
| start_rotation | speed | Unbounded rotational movement |
| stop_movement | None | Stops movement |
| stop_rotation | None | Stops rotation |

### D. Remote / Laptop Programming Tasks

While the Jetbot ran on an equipped Jetson Orin Nano SPC, it still lacked the computational efficiency to handle our demanding YoloE CV Model [2]. Thus, our project required a second subtask of programming remote functions on a laptop, equipped with a powerful NVIDIA 5080 graphics card with 16GB of VRAM. This system ran a web-application frontend built with React, the YoloE annotation / telemetry streaming server, and the full suite of Agentic LLMs powered by Google ADK.

Upon startup, the Jetbot would automatically poll and connect to the YoloE socket running on the laptop given an SSH tunnel was open. From there, it would stream the camera feed captured by the Arducam and receive the annotated JSON labels that will be used in the visual capabilities - scan and center. The labels used in the YoloE web server were set either by the user or the Agents based on the task at hand.

The web server was constructed to allow a user oversight into the system, real-time telemetry data (ultrasonic sensor, detected items, motor speed), and a chat window to interact with agents. In this web interface, a user may also use the unbounded movement functions to drive the car via the arrow keys, as well as an emergency stop. This helped develop trust, visibility, and control for the system.

Lastly, the Agentic LLM system was built by Google ADK and was inspired by the PLAN-SENSE-ACT paradigm, with a dedicated agent corresponding to each function. We built the following agents:

- DIRECTOR: corresponded to the PLAN primitive. The director created a long term, natural language description of the task proposed by the user and kept track of when a plan should be marked complete. Had means of intervention to halt the PILOT or OBSERVER agents.

- PILOT: corresponded to the ACT primitive. The PILOT had primary control of movement functions and would move as informed by the global DIRECTOR plan, and visual information conveyed to it by the OBSERVER.
- OBSERVER: corresponded to the SENSE primitive. Worked in parallel with the PILOT and informed it of various visual contexts relevant to the overall mission, such as where an item is, if there is free roam from the ultrasonic readings, or if there is no target item to be seen.

The DIRECTOR agent ran overseeing the other two agents, which ran in parallel across two executors. This allowed for the PILOT to move while the OBSERVER viewed the world around it and could alarm the PILOT of any new readings. This paradigm was effective and also closely mirrored the way multiple humans may teleoperate a complex system.

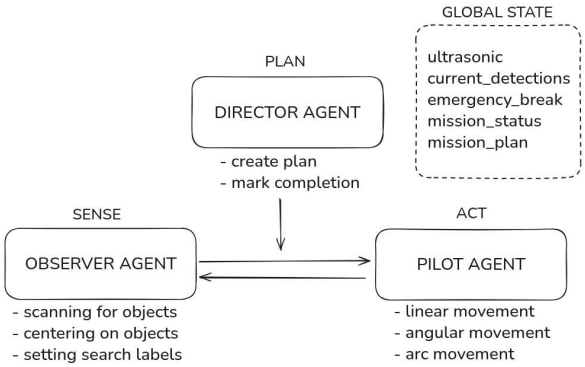


Fig. 2 Overview of the Agentic LLM flow and interactions.

### *E. Assembly and System Design*

After assembly, edge programming, and remote laptop programming was finished, the system needed to be put together. Both the edge system and remote system communicated via localhost ports over an SSH tunnel, which allowed them to essentially share a port over the network. This allowed for the following functions:

- User teleoperation of the robot via the Web UI
- Jetbot/Laptop communication on the telemetry websocket and YoloE socket
- Agent remote execution of robot controls (core function of the system)

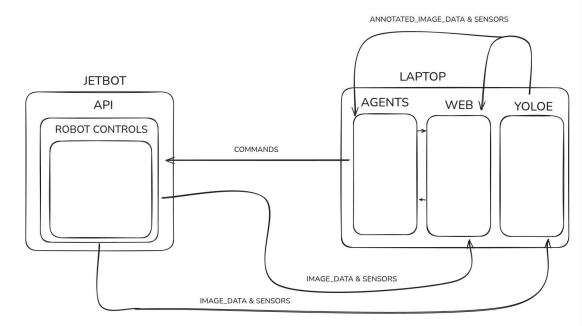


Fig. 3 Overview of the System Design

## III. Results

Overall, the final result of the agentic system was deemed a success by the team. While this project was AI-related, there was no feasible way for us to measure task accuracy procedurally which posed issues for us to assess system performance. Given that tasks are not consistently reproducible or automatically detectable, instead we ran different "scenario tests" to gauge its generalization capabilities. The ultimate goal of this system was for it to reliably complete a task it has never been explicitly programmed to do, with the help of AI.

We ran the following scenarios for generalization testing that involved spatial awareness, movement understanding, and visual capabilities

- "Draw a square with side lengths of 1 meter"
- "Draw a circle with radius of 0.5 meters"
- "Help me find something to drink"
- "Find my friend who has fallen and drive up to him"

These scenarios were very interesting to calibrate with, as each succeeded at testing both the tools we created, and generalizable concepts. In the case of the drink test, the models successfully generalized the task in the scan function, looking for multiple labels (bottle, water bottle, can, soda, juice) rather than just "drink," showcasing how the reasoning capabilities of LLMs made this task more achievable. As for the most complex task of locating a fallen person, the system's understanding

of aspect-ratio via the labeling data (vertical has taller bounding boxes than wider ones) helped it successfully identify *which* person to approach. While these tasks can be trained in a RL environment, that system would be able to perform well in *only* that subset of tasks. This system instead aimed for generality, and we feel it has succeeded.

The system's full codebase is available on GitHub [3], with a more in depth demo hosted on YouTube [4].

## IV. Discussion

The operation of our robot, Hot Dog, demonstrates the capabilities of combining a Jetbot platform with an agentic LLM system capable of performing generalized decision-making tasks. Overall the main behaviors that we implemented worked reliably during testing and behaved consistently when invoked by the director and pilot agents. The teleoperation layer, supported by our React-based web interface also proved stable as it allowed our real-time visualization to take full advantage of its surroundings. YOLOE labels, ultrasonic sensor data, and manual override controls helped greatly during debugging and demonstrations.

While testing did better than expected among the group, we still faced several challenges that became more apparent throughout the demo. Hot Dog's vision labeling was very inconsistent at the start, and through multiple sessions of debugging and further testing we found a solution to an issue we didn't hypothesize could happen. The inclusion of the ultrasonic sensors on the front of the Jetbot was being read during its YOLOE visualization testing as a "cup" label. The team specifically chose to use a "water bottle" as one of Hot Dog's first open-ended tasks and since both are similar, this led to the malfunctioning of certain operations the agentic system was dealing with. It would recognize the ultrasonic sensor during reasoning and immediately conclude it reached its destination since it was within range of the Jetbot. We quickly fixed this issue by attaching a piece of material to cover the ultrasonic sensor from the robot's view.

Our next challenge dealt with having multiple of the same type of object within the view of Hot Dog. This led to the system essentially picking and choosing which object to go after. We dealt with this issue during our test runs and even our demonstration for asking Hot Dog to look for my "backpack". This issue though was more out of our scope than the previous and might not even be considered an issue for certain scenarios. It all depends on the surroundings and would require further testing and potential next steps for our project we simply could not do within the deadline.

Our final challenge dealt with the robot exhibiting a slight mechanical imbalance due to the uneven weight distribution on one side. This isn't too noticeable on certain distance speeds, but the higher and further it went, the easier it was to spot. Due to time constraints and not being able to reconfigure the entire mechanical aspect of the robot to fix this, we had to deal with this problem. As a result, even when executing a simple move operation, the robot tended to drift and required additional corrective rotation before aligning with its intended target. To combat this, we prioritized using a slower distance speed when running our tests and demonstrations.

If we had more time, we could evaluate our system through structured experiments that could help quantify the physical reliability of the robot and its intelligence. This could include potential heavy 3D modeling before crafting the robot, decision making accuracy tests for the LLM, controlling precise measurements for rotational accuracy, etc., and even user study evaluations for how they would interact with Hot Dog. Together these evaluations would provide a more complete picture of the system's overall capabilities for agentic reasoning and mechanical design of the robot.

## V. Conclusion

Hot Dog demonstrates the potential of VLA systems in using an LLM to perform generalized, open-ended tasks by using its vision capabilities and primitive motor functions. Therefore, the performance of the task completion is merely

limited by the physical constraints of the robot. By adding new sensory tools the agent has access to more context and observations which can be used for a better spatial understanding leading to more quality steps to solve the task given.

To improve the robot's operations, our focus would be on further testing and refining its mechanical motion ability. By adding closed-loop motor feedback not only will it be able to have more reliable and precise motion to solve our weight imbalance problem, but it would also provide better insight as to how to further calibrate and balance the chassis and mount layout.

## VI. Contributions

Dylan: spearheaded project idea, vision, and development. Worked primarily on the Agentic LLM system via Google ADK, and handling networking between edge and remote compute systems. Oversaw implementation of edge robotic functions and API exposition.

Richard: spearheaded hardware assembly and systems integration for the robot. Led decision making on the component section ensuring compatibility across all subsystems. Assembled the full platform from individual components, including soldering and pin mapping for all electronics. Developed the frontend interface in TypeScript and React, enabling live camera views.

Diego: Developed several APIs that exposed the robot's movement logic and high level behaviors for the system to call them through HTTP. Implemented the telemetry connection between the front-end and the Jetbot, enabling manual control of the bot, adjusting of robot speed, and display or overwrite of labels for the agent to utilize. Designed some additional pieces for the robot and printed them.

Anthony: Developed initial testing with YOLOE visualization model for the robot with help from Diego, the YOLOE label prompting that allows users and the agentic system to specify which labels to target during its tasks, and the main user interface/user experience for the website with help from everyone on the team.

## VII. References


[1] NVIDIA-AI-IOT. "JetBot." *JetBot Documentation*, jetbot.org/master/index.html. Accessed 29 Aug. 2025.

[2] Ultralytics. "YOLOE." *Ultralytics Documentation*, docs.ultralytics.com/models/yoloe/. Accessed 29 Aug. 2025.

[3] Team Hot Dog. "EGN4060 / final-project." *GitHub Repository*, github.com/ENG4060-C/final-project. Accessed: Dec. 4, 2025.

[4] Team Hot Dog. "EGN4060 - Final Video Demo." *youTube*, www.youtube.com/watch?v=sI2yJACUfRg. Accessed: Dec. 4, 2025.